PX4 + ROS 2 con px4_examples: referencia técnica

El x500, los mensajes de PX4 campo por campo y los 18 ejemplos con su código

Imad Jared Cabrera Trejo

Vespo UAV Drones Autónomos

11 de noviembre de 2026



Agenda



1. PX4 y el x500
2. Cómo habla ROS 2 con PX4
3. Los mensajes de PX4
4. Ejemplos de solo lectura
5. Un topic propio
6. Offboard: nosotros calculamos
7. Comandos: PX4 vuela

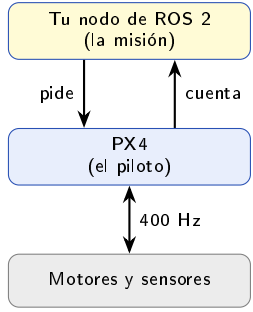
Contenido

1. PX4 y el x500
2. Cómo habla ROS 2 con PX4
3. Los mensajes de PX4
4. Ejemplos de solo lectura
5. Un topic propio
6. Offboard: nosotros calculamos
7. Comandos: PX4 vuela

Idea central

PX4 es el piloto automático: el programa que corre en la controladora de vuelo y mantiene al dron en el aire. **ROS 2** es donde escribimos *la misión*: qué hacer, a dónde ir, cuándo aterrizar.

Nosotros **no controlamos los motores**: le pedimos cosas a PX4 ("ve a este punto", "aterriza") y leemos lo que nos cuenta (posición, batería, modo).



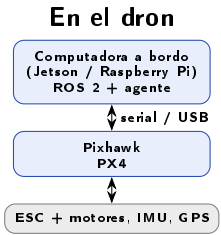
¿Qué es PX4?

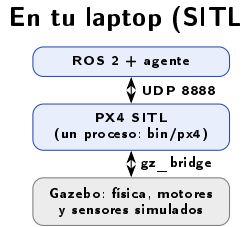
- Un **autopiloto de código abierto** (BSD) para drones, aviones, VTOL, rovers y más.
- Corre en una **controladora de vuelo** (Pixhawk 6C, 6X, Cube...) con un sistema operativo de tiempo real (NuttX).
- Por dentro son **módulos** independientes: sensores, estimador, controladores, `commander` (modos y armado), `navigator` (misiones), registro...
- La misma base de código corre en **simulación** (SITL) en tu computadora.
- Usamos **PX4 v1.17.0** (fijado en `vespo.toml`); `px4_msgs` tiene que coincidir.



Pixhawk 6C. Docs de PX4 (CC BY 4.0)

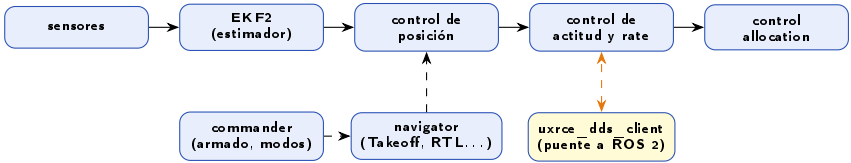
Real contra simulación





Tu nodo **no sabe** si habla con un dron real o con la simulación: los topics son los mismos. Por eso todo se prueba primero en SITL.

Por dentro: módulos que se hablan por uORB



- **uORB**: el "ROS interno" de PX4. Cada módulo publica y se suscribe a *topics* uORB (`vehicle_local_position`, `battery_status`...).
- El módulo `uxrce_dds_client` copia algunos de esos topics a ROS 2 y trae otros de vuelta.

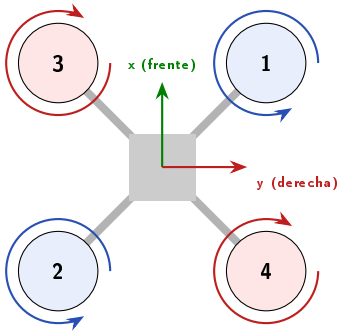
El x500

- Cuadricóptero de Holybro, marco en **X**, ~500 mm de motor a motor en diagonal.
- Es el modelo por defecto de PX4 en Gazebo (`gz_x500`, airframe 4001).
- Variantes de PX4 con sensores: `x500_lidar_2d`, `x500_mono_cam_down`, `x500_depth`...
- En SITL flota (hover) con ~60 % de empuje (`MPC_THR_HOVER = 0.60`).



Holybro X500 V2. Documentación de PX4 (CC BY 4.0)

Los motores: número, posición y giro

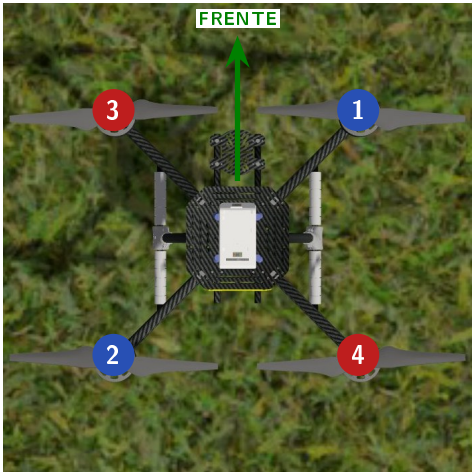


Del airframe `4001_gz_x500` (en metros, marco FRD):

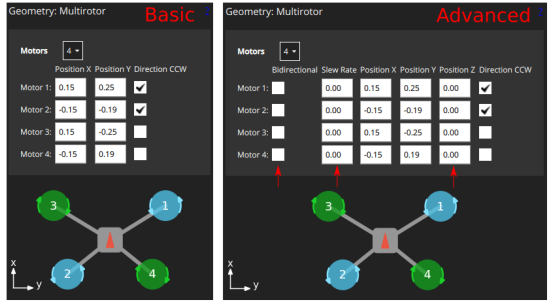
Motor	x	y	Giro
1	+0.13	+0.22	antihorario
2	−0.13	−0.20	antihorario
3	+0.13	−0.22	horario
4	−0.13	+0.20	horario

Si en el dron real dos motores están cruzados, **se voltea al despegar**: para eso es `motor_test`.

Los motores en el x500 de la simulación



Render propio de Gazebo, vista desde arriba



QGroundControl. Docs de PX4 (CC BY 4.0)

Mismo orden en el modelo de Gazebo, en el airframe y en QGC: azul antihorario, rojo horario.

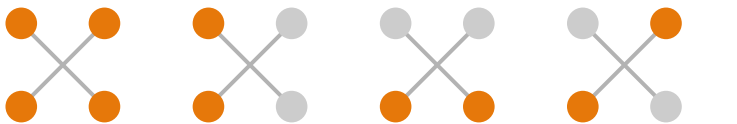
¿Por qué la mitad gira al revés?

Cada hélice que gira empuja hacia arriba **y** tuerce el cuerpo en sentido contrario (par de reacción).

- Dos giran a un lado y dos al otro: los pares se **cancelan** y el dron no gira solo.
- Para **girar** (yaw), PX4 acelera un par y frena el otro: el empuje total es el mismo, pero los pares ya no se cancelan. Acelerar los antihorarios (1, 2) tuerce el cuerpo en sentido horario: yaw a la derecha.

En el airframe esto es `CA_ROTORn_KM`: $+0.05$ gira antihorario, -0.05 horario. El *control allocation* lo usa para repartir el giro.

Cómo se mueve un cuadricóptero



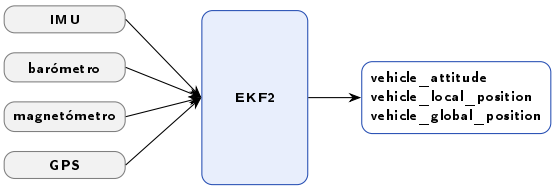
Un multicóptero solo puede **inclinarse** y **empujar hacia "arriba"** (de su cuerpo). Para avanzar se inclina: parte del empuje apunta hacia adelante.

Los sensores

Sensor	Mide	Frecuencia típica	Topic de ROS 2
Giroscopio	velocidad de giro (rad/s)	cientos de Hz	`/fmu/out/sensor_combined`
Acelerómetro	aceleración (m/s^2)	cientos de Hz	`/fmu/out/sensor_combined`
Barómetro	presión $\rightarrow$ altura	~50 Hz	(no expuesto)
Magnetómetro	campo magnético $\rightarrow$ norte	~50 Hz	(no expuesto)
GPS	latitud, longitud, altitud	5–10 Hz	`/fmu/out/vehicle_gps_position`

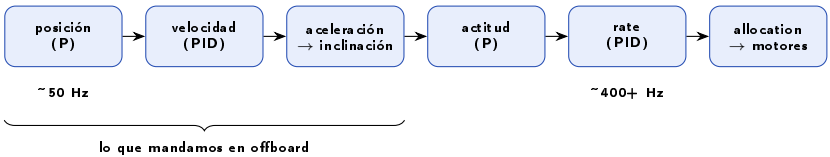
En SITL los **simula Gazebo** con ruido; PX4 los recibe por `gz_bridge` como si fueran reales. Por eso el GPS de la simulación "se mueve" unos centímetros aunque el dron esté quieto.

EKF2: de sensores a “dónde estoy”



- Un **filtro de Kalman extendido**: combina sensores que se equivocan de formas distintas.
- La IMU es rápida pero se desvía; el GPS es lento pero no se desvía: juntos dan una posición buena y rápida.
- Lo que leemos en ROS 2 (`vehicle_local_position...`) es la **estimación**, no el sensor crudo.

La cadena de control (en cascada)



- Cada lazo le da su *setpoint* al siguiente: el de posición dice "ve a 2 m/s", el de velocidad dice "inclínate 10°", el de actitud dice "gira a 1 rad/s"...
- Los lazos de adentro son **más rápidos** que los de afuera.
- En offboard nosotros entramos **arriba**: posición, velocidad o aceleración.

Modos de vuelo (`nav_state`)

#	Modo	Quién manda	#	Modo	Quién manda
0	Manual	piloto	4	Hold (Loiter)	PX4: quieto en un punto
15	Stabilized	piloto (nivela solo)	17	Takeoff	PX4: despega
1	Altitude	piloto, PX4 sostiene altura	18	Land	PX4: aterriza
2	Position	piloto, PX4 sostiene posición	5	Return (RTL)	PX4: a casa
3	Mission	PX4: plan de QGC	21	Orbit	PX4: círculo
14	**Offboard**	**tu nodo**	12	Descend	PX4: baja sin GPS

- En los ejemplos se compara con las constantes: `VehicleStatus::NAVIGATION_STATE_OFFBOARD`.
- `flight_mode_monitor` los imprime cada vez que cambian.

Dos maneras de mandar

Offboard

Tu nodo calcula un **setpoint** 10–20 veces por segundo (posición, velocidad...).

- Control fino: trayectorias, seguir algo.
- Si el nodo se cae, PX4 entra en **failsafe**.
- `takeoff_mission`, `square_mission`, `orbit_mission`, `figure_eight`, `velocity_control`.

Comandos

Tu nodo manda **órdenes**: despega, ve a, orbita, regresa. PX4 vuela con sus modos.

- Menos código, más seguro: si el nodo se cae, PX4 sigue.
- Menos control fino.
- `orbit_command`, `goto_mission`, `return_home`, `change_speed`, `set_home`.

Armar

Armar = darle permiso a los motores de girar. Antes, PX4 corre los **preflight checks**:

- sensores calibrados y coherentes, EKF2 convergido,
- GPS con fix (si el modo lo necesita), batería suficiente,
- enlace con la estación de tierra (`NAV_DLL_ACT`),
- el modo pedido se puede usar ahora.

Si algo falla, rechaza el armado y lo dice en la consola y en QGC.

consola de PX4

```
WARN Preflight Fail:
ekf2 missing data
WARN Preflight Fail:
No connection to the GCS
WARN Arming denied: Resolve
system health failures first
```

Al tocar tierra, PX4 **desarma solo** (unos segundos después).

Failsafes

Cuando algo sale mal, PX4 toma el control y hace una acción configurada:

Situación	Parámetro	Acción típica
Se dejan de recibir setpoints de offboard	`COM_OF_LOSS_T`, `COM_OBL_RC_ACT`	Hold / Return / Land
Se pierde la estación de tierra	`NAV_DLL_ACT`	Return
Batería baja / crítica	`BAT_LOW_THR`, `COM_LOW_BAT_ACT`	aviso / Return / Land
Se sale de la geocerca	`GF_ACTION`	aviso / Hold / Return
Se pierde el control remoto	`NAV_RCL_ACT`	Return

`/fmu/out/failsafe_flags` trae una bandera por cada cosa que PX4 vigila; `flight_mode_monitor` las muestra.

Parámetros de PX4 (no son los de ROS)

PX4 tiene ~1000 parámetros propios: ganancias, límites, alturas. Se ven y cambian en QGroundControl (Vehicle Setup → Parameters) o en la consola `pxh>`:

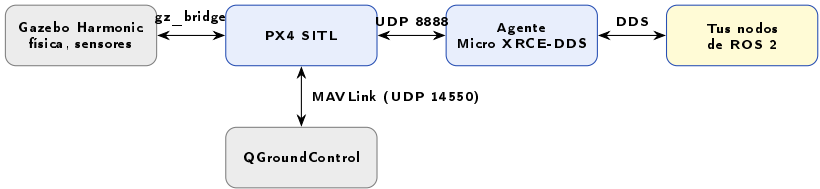
```
pxh> param show MIS_TAKEOFF_ALT        # altura del modo Takeoff
pxh> param show MPC_XY_VEL_MAX         # velocidad horizontal máxima
pxh> param set RTL_RETURN_ALT 15       # altura a la que regresa Return
pxh> commander status                  # estado general
pxh> listener vehicle_local_position   # leer un topic uORB
```

Código 1: En la terminal donde corre PX4 SITL

Ojo

No confundir con los parámetros de ROS 2 de nuestros nodos (`-ros-args -p radius:=8.0`): esos son de tu programa, no de PX4.

La simulación completa



- `vespo up -sim` levanta todo esto: el agente, PX4 + Gazebo y QGroundControl.
- **Lockstep**: PX4 avanza al paso de Gazebo. Si la simulación va lenta, PX4 también; tu nodo (con el reloj de la pared) no.

Contenido



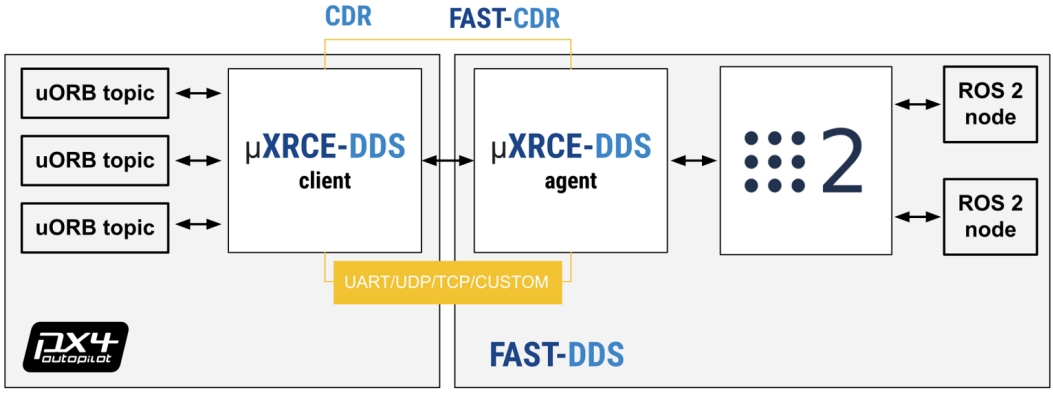
1. PX4 y el x500
2. Cómo habla ROS 2 con PX4
3. Los mensajes de PX4
4. Ejemplos de solo lectura
5. Un topic propio
6. Offboard: nosotros calculamos
7. Comandos: PX4 vuela

El puente: uXRCE-DDS



- DDS "completo" es pesado para una controladora: PX4 habla **XRCE-DDS**, la versión para microcontroladores.
- El **agente** traduce a DDS normal: del otro lado ROS 2 ve topics comunes y corrientes.
- Sin agente corriendo, **no hay ningún topic de PX4** en ROS 2.
- SITL: `MicroXRCEAgent udp4 -p 8888`. Dron real: por serial (`vespo up -real`).

El diagrama oficial



Arquitectura de uXRCE-DDS. Documentación de PX4 (CC BY 4.0)

A la izquierda los topics uORB dentro de PX4; en medio el cliente (en PX4) y el agente (en la computadora) unidos por UART, UDP o TCP; a la derecha los nodos de ROS 2.

/fmu/out y /fmu/in

/fmu/out/...: PX4 nos cuenta

```
vehicle_status_v1
vehicle_local_position_v1
vehicle_global_position
vehicle_attitude
sensor_combined
vehicle_gps_position
battery_status_v1
home_position_v1
failsafe_flags
vehicle_command_ack
```

/fmu/in/...: le pedimos a PX4

```
offboard_control_mode
trajectory_setpoint
vehicle_command
```

Solo existen los topics que PX4 decide exponer: la lista está en `src/modules/uxrce_dds_client/dds_topics.yaml` de PX4 (con su frecuencia máxima).

dds_topics.yaml

```
publications:
  - topic: /fmu/out/battery_status
    type: px4_msgs::msg::BatteryStatus
    rate_limit: 1.            # a 1 Hz como mucho
  - topic: /fmu/out/vehicle_attitude
    type: px4_msgs::msg::VehicleAttitude
subscriptions:
  - topic: /fmu/in/trajectory_setpoint
    type: px4_msgs::msg::TrajectorySetpoint
  - topic: /fmu/in/vehicle_command
    type: px4_msgs::msg::VehicleCommand
```

Código 2: Un pedazo del archivo de PX4 v1.17

Si un ejemplo no recibe nada, lo primero es revisar que el topic esté aquí y que el nombre sea exacto (incluido el sufijo de versión).

Los sufijos _v1: versiones de mensajes

Cuando PX4 cambia los campos de un mensaje, sube su `MESSAGE_VERSION` y el topic cambia de nombre:

Mensaje	versión	topic
`VehicleStatus`	1	`vehicle_status_v1`
`VehicleLocalPosition`	1	`vehicle_local_position_v1`
`BatteryStatus`	1	`battery_status_v1`
`HomePosition`	1	`home_position_v1`
`VehicleGlobalPosition`	0	`vehicle_global_position`
`VehicleAttitude`	0	`vehicle_attitude`

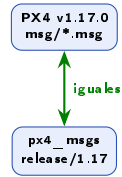
Versión 0 = sin sufijo.

Ojo

Si te suscribes a `/fmu/out/vehicle_status` (sin `_v1`) **no llega nada** y no hay ningún error: simplemente nadie publica en ese nombre.

px4_msgs tiene que coincidir con PX4

- `px4_msgs` es un paquete de ROS 2 con los **mismos .msg** que tu versión de PX4.
- Si no coinciden, el agente no puede traducir: los topics "existen" pero los datos salen mal o no llegan.
- Por eso `vespo.toml` fija la versión de PX4 (v1.17.0) y `vespo.repos` el commit de `px4_msgs` (`release/1.17`).
- `vespo px4 update` mueve los dos juntos.



QoS: cómo se entregan los mensajes

Cada publisher y subscriber de ROS 2 tiene una **calidad de servicio** (QoS). PX4 publica con:

```
rclcpp::QoS(1).best_effort().transient_local()
```

Parte	Significa	Por qué
`QoS(1)`	guardar solo el último mensaje	solo importa el dato más reciente
`best_effort`	si se pierde uno, no se reenvía	a 100 Hz el siguiente llega enseguida
`transient_local`	al conectarte recibes el último que se publicó	no esperar al siguiente

Ojo

Un subscriber `reliable` **no recibe nada** de un publisher `best_effort`. Por eso todos los ejemplos crean su suscripción con el mismo QoS que PX4.

Compatibilidad de QoS

Publisher	Subscriber	¿Se conectan?
best_effort	best_effort	✓
reliable	best_effort	✓
reliable	reliable	✓
best_effort	reliable	× (nunca llega nada)

- Regla: el subscriber puede pedir **menos** que lo que ofrece el publisher, nunca más.
- Entre nodos nuestros (`battery_publisher` → `battery_subscriber`) usamos el QoS por defecto (reliable), porque no hace falta otra cosa.
- `ros2 topic info -v /fmu/out/vehicle_status_v1` muestra el QoS de cada lado.

Timestamps: microsegundos desde que arrancó PX4

Todos los mensajes de PX4 empiezan con `uint64 timestamp`: tiempo desde que arrancó el sistema, en **microsegundos**.

```
uint64_t now_us() {
  return get_clock()->now().nanoseconds() / 1000;   // ns -> us
}
msg.timestamp = now_us();
```

Código 3: Así ponen el timestamp los ejemplos al publicar

- Si un mensaje recibido tiene `timestamp == 0`, **todavía no ha llegado ninguno** (los mensajes nacen con todos sus campos en 0). `telemetry_listener` lo usa para avisar.
- PX4 sincroniza su reloj con el del agente; en la práctica basta con mandar el tiempo actual.

Explorar desde la terminal

```
ros2 topic list | grep fmu                              # qué expone PX4
ros2 topic echo /fmu/out/vehicle_status_v1 --once       # un mensaje
ros2 topic hz /fmu/out/sensor_combined                  # qué tan rápido llega
ros2 interface show px4_msgs/msg/VehicleCommand         # los campos de un tipo
ros2 topic info -v /fmu/out/vehicle_local_position_v1 # QoS de cada lado
```

Código 4: Con el agente y PX4 corriendo

ros2 topic echo /fmu/out/vehicle_status_v1 -field nav_state

```
14
--
14
```

Contenido

1. PX4 y el x500
2. Cómo habla ROS 2 con PX4
3. Los mensajes de PX4
4. Ejemplos de solo lectura
5. Un topic propio
6. Offboard: nosotros calculamos
7. Comandos: PX4 vuela

Anatomía de un .msg

```
uint32 MESSAGE_VERSION = 0       # constante

uint8 VEHICLE_CMD_RESULT_ACCEPTED = 0
uint8 VEHICLE_CMD_RESULT_DENIED = 2

uint64 timestamp      # campo: tipo nombre
uint32 command
uint8 result
```

Código 5: px4_msgs/msg/VehicleCommandAck.msg (recortado)

- Cada línea es **tipo nombre** (un campo) o **tipo NOMBRE = valor** (una constante).
- `colcon build` genera una **estructura de C++** con esos campos.
- Las constantes quedan como miembros estáticos: `VehicleCommandAck::VEHICLE_CMD_RESULT_DE`

Tipos de los .msg y su tipo en C++

En el .msg	En C++	Para qué
`bool`	`bool`	banderas: `xy_valid`, `failsafe`
`uint8`	`uint8_t` (0..255)	modos y estados: `nav_state`, `arming_state`
`uint32`	`uint32_t`	ids de comando: `command`
`uint64`	`uint64_t`	tiempos en µs: `timestamp`
`float32`	`float`	casi todo: posición, velocidad, ángulos
`float64`	`double`	latitud y longitud (necesitan precisión)
`float32[3]`	`std::array<float, 3>`	vectores: `position`, `velocity`
`float32[4]`	`std::array<float, 4>`	cuaterniones: `q`

`float` tiene ~7 dígitos: alcanza para metros, no para grados de latitud (47.3979711° necesita 9). Por eso `lat/lon` son `float64`.

Del .msg al #include

```
#include "px4_msgs/msg/vehicle_status.hpp"        // archivo: vehicle_status
using px4_msgs::msg::VehicleStatus;               // tipo: VehicleStatus

VehicleStatus status;                              // una variable de ese tipo
if (status.nav_state == VehicleStatus::NAVIGATION_STATE_OFFBOARD) { ... }
```

Código 6: El archivo va en snake_case, el tipo en CamelCase

- `px4_msgs::msg::` es el *namespace* (paquete `px4_msgs`, carpeta `msg`).
- `using` evita escribir el nombre completo cada vez (como `from px4_msgs.msg import ...` en Python).

VehicleStatus: el estado general

Campo	Tipo	Qué es
`arming_state`	uint8	1 desarmado, 2 armado
`nav_state`	uint8	el modo activo (14 = Offboard)
`nav_state_user_intention`	uint8	el modo que *pidió* el usuario
`failsafe`	bool	¿hay un failsafe activo?
`pre_flight_checks_pass`	bool	¿se puede armar?
`gcs_connection_lost`	bool	¿se perdió QGC?
`armed_time`	uint64	cuándo armó (µs)

Topic: `/fmu/out/vehicle_status_v1`.

Es el **heartbeat** de PX4: casi todos los ejemplos lo leen para saber si ya armó y en qué modo está.

Si `nav_state` $\neq$ `nav_state_user_intention`, PX4 está haciendo otra cosa que lo que pediste (p. ej. un failsafe).

VehicleLocalPosition: dónde estoy (en metros)

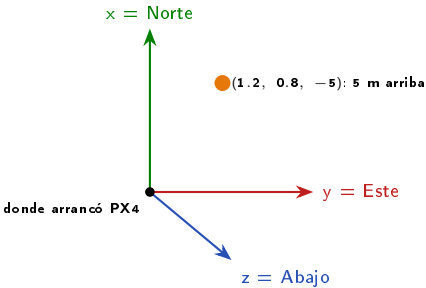
Campo	Unidad	Qué es
`x, y, z`	m	norte, este, abajo (NED)
`vx, vy, vz`	m/s	velocidad NED
`ax, ay, az`	m/s^2	aceleración NED
`heading`	rad	yaw, $-\pi..\pi$, 0 = norte
`xy_valid, z_valid`	bool	¿la estimación sirve?
`dist_bottom`	m	distancia al suelo (si hay sensor)

Topic:
`/fmu/out/vehicle_local_position_v1`.

Ojo

El origen es donde **arrancó PX4** (no donde está el dron ahora), y z es hacia **abajo**: la altura es `-z`.

NED: el marco de VehicleLocalPosition



- **N**orth, **E**ast, **D**own: el estándar de la aviación.
- Mandar $z = -5$ es pedir **5 m de altura**; $z = +5$ es pedir 5 m bajo tierra.
- ROS usa ENU (x este, y norte, z arriba). PX4 usa NED en todos sus mensajes: no hay que mezclarlos.

VehicleGlobalPosition: dónde estoy (en el planeta)

Campo	Tipo	Qué es
`lat`, `lon`	float64	grados
`alt`	float32	m sobre el nivel del mar (AMSL)
`lat_lon_valid`, `alt_valid`	bool	¿sirve?
`eph`, `epv`	float32	error estimado horiz./vert. (m)

Topic: `/fmu/out/vehicle_global_position`.

Es la estimación del EKF2 (no el GPS crudo). Los comandos de PX4 (`NAV_TAKEOFF`, `DO_REPOSITION`, `DO_SET_HOME`) piden coordenadas y **altitud AMSL**, no metros locales.

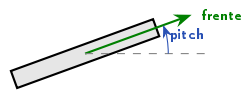
VehicleAttitude: la orientación como cuaternión

Topic: `/fmu/out/vehicle_attitude`. Un solo campo importante:

$$\texttt{float32[4] q} \quad = [w,\ x,\ y,\ z]$$

la rotación del cuerpo (FRD) a NED.

- Un **cuaternión** describe una rotación con 4 números sin los problemas de los ángulos de Euler (*gimbal lock*).
- Cuidado: PX4 lo da como $[w, x, y, z]$; ROS (`geometry_msgs`) usa $[x, y, z, w]$.



roll, pitch, yaw = 3 ángulos
q = la misma rotación en 4 números

De cuaternión a roll, pitch y yaw

```
// PX4 da el cuaternión en orden [w, x, y, z] (ROS usa [x, y, z, w]).
const float w = attitude_.q[0];
const float x = attitude_.q[1];
const float y = attitude_.q[2];
const float z = attitude_.q[3];
const float roll = std::atan2(2.0f * (w * x + y * z), 1.0f - 2.0f * (x * x + y * y));
// asin solo acepta valores en [-1, 1]; std::clamp evita NaN por redondeo.
const float pitch = std::asin(std::clamp(2.0f * (w * y - z * x), -1.0f, 1.0f));
const float yaw = std::atan2(2.0f * (w * z + x * y), 1.0f - 2.0f * (y * y + z * z));
```

Código 7: telemetry_listener.cpp

Con el dron nivelado: roll $\approx$ pitch ≈ 0 y yaw = hacia dónde apunta la nariz (igual a `heading` de la posición local).

SensorCombined: la IMU

Topic: `/fmu/out/sensor_combined`, a cientos de Hz.

Campo	Unidad	Qué es
`gyro_rad[3]`	rad/s	velocidad de giro en x, y, z (FRD)
`accelerometer_m_s2[3]`	m/s^2	aceleración medida (FRD)
`gyro_integral_dt`	µs	periodo de muestreo
`accelerometer_clipping`	bits	¿se saturó algún eje?

Quieto en el suelo el acelerómetro marca $(0, 0, -9.81)$: mide la fuerza que lo **sostiene** (el suelo empuja hacia arriba, que en FRD es z negativa), no la gravedad.

SensorGps: el GPS crudo

Topic: `/fmu/out/vehicle_gps_position` (versión 0: sin sufijo). Es lo que dice el receptor, **antes** del EKF2. Sirve para saber si ya se puede volar con GPS (`gps_status`).

Campo	Qué es
`fix_type`	0–1 sin fix, 2 = 2D, 3 = 3D, 4 = DGPS, 5 = RTK float, 6 = RTK fixed
`satellites_used`	satélites usados en la solución
`eph`, `epv`	error estimado horizontal / vertical (m)
`hdop`	qué tan buena es la geometría de los satélites (<2 es buena)
`latitude_deg`, `longitude_deg`	posición (float64)
`jamming_state`	0 desconocido, 1 OK, 2 mitigado, 3 interferencia detectada

BatteryStatus

Campo	Unidad	Qué es (inválido)
`connected`	bool	¿hay batería?
`voltage_v`	V	voltaje total (0)
`current_a`	A	corriente (-1: sin sensor, como en SITL)
`remaining`	0..1	carga restante (-1)
`time_remaining_s`	s	hasta vacía (NaN)
`cell_count`	–	celdas (4S = 4)
`warning`	–	0 nada, 1 baja, 2 crítica, 3 emergencia

Topic: `/fmu/out/battery_status_v1`, a **1 Hz** (lo limita `dds_topics.yaml`).

Ojo

Muchos campos usan un valor "imposible" para decir **no sé**: -1 o NaN. Revísenlo antes de usar el dato (`battery_publisher` lo hace).

FailsafeFlags y HomePosition



`/fmu/out/failsafe_flags`: ~26 banderas (casi todas `bool`), una por cada cosa que PX4 vigila:

- `local_position_invalid`, `global_position_invalid`
- `home_position_invalid`
- `offboard_control_signal_lost`
- `manual_control_signal_lost`, `gcs_connection_lost`
- `battery_warning`, `geofence_breached`...

`/fmu/out/home_position_v1`: "casa", a donde va Return.

- `lat`, `lon`, `alt`: en GPS
- `x`, `y`, `z`: en NED local
- `valid_hpos`, `valid_alt`
- `update_count`: sube cada vez que casa cambia

PX4 la fija **donde arma** y solo la publica cuando cambia.

VehicleCommand: pedirle algo a PX4

Campo	Tipo	Qué es
`command`	uint32	qué comando (`VEHICLE_CMD_...`)
`param1..param4`	float32	sus parámetros
`param5, param6`	**float64**	normalmente lat, lon
`param7`	float32	normalmente altitud
`target_system`	uint8	a quién va (1)
`target_component`	uint8	1 (0 = todos)
`source_system/component`	–	quién lo manda (1, 1)
`from_external`	bool	**true**: viene de afuera de PX4

Topic: `/fmu/in/vehicle_command`.

Son los comandos de **MAVLink** (los mismos que manda QGC): cada uno define qué significa cada parámetro.

Los comandos que usan los ejemplos

Comando	Parámetros	Ejemplos
`DO_SET_MODE`	p1 = 1; p2 = 6 offboard, o p2 = 4 + p3 = 3 Hold	casi todos
`COMPONENT_ARM_DISARM`	p1 = 1 armar, 0 desarmar	todos los que vuelan
`NAV_TAKEOFF`	p5, p6 = lat, lon; p7 = alt AMSL	orbit_command, goto...
`NAV_LAND`	—	square, orbit, figure_eight...
`NAV_RETURN_TO_LAUNCH`	—	orbit_command, return_home...
`DO_REPOSITION`	p1 = vel, p2 = 1, p4 = yaw, p5..7 = lat, lon, alt	goto_mission, change_speed
`DO_ORBIT`	p1 = radio, p2 = vel, p5..7 = centro	orbit_command
`DO_CHANGE_SPEED`	p1 = 1 (suelo), p2 = m/s	change_speed
`DO_SET_HOME`	p1 = 0, p5..7 = lat, lon, alt	set_home
`ACTUATOR_TEST`	p1 = 0..1, p2 = s, p5 = motor	motor_test

VehicleCommandAck: ¿me hizo caso?

result	Significa
0	ACCEPTED: aceptado y ejecutado
1	TEMPORARILY_REJECTED: ahora no
2	DENIED: no, nunca
3	UNSUPPORTED: no lo conozco
4	FAILED: lo intenté y falló
5	IN_PROGRESS: trabajando
6	CANCELLED

Topic: `/fmu/out/vehicle_command_ack`. Trae `command` (cuál) y `result`.

goto_mission

```
PX4 rechazó el comando 400
(result=1)
```

400 = armar; 1 = “ahora no” (p. ej. falló un preflight check).

OffboardControlMode: el heartbeat de offboard

Campo (bool)	Controlar...
`position`	posición
`velocity`	velocidad
`acceleration`	aceleración
`attitude`	actitud
`body_rate`	velocidad de giro
`thrust_and_torque`	empuje y par
`direct_actuator`	los motores directo

Topic: `/fmu/in/offboard_control_mode`.

- Dice **qué tipo** de setpoint vamos a mandar.
- Hay que publicarlo **continuamente** (>2 Hz): si deja de llegar, PX4 sale de offboard (failsafe).
- Los ejemplos solo usan `position`, `velocity` y `acceleration`.

TrajectorySetpoint: a dónde ir

Campo	Unidad	
`position[3]`	m	NED
`velocity[3]`	m/s	NED
`acceleration[3]`	m/s^2	NED
`yaw`	rad	$-\pi..\pi$
`yawspeed`	rad/s	

Topic: `/fmu/in/trajectory_setpoint`.

Lo que va en NaN, PX4 no lo usa:

- posición + NaN en velocidad = ir al punto;
- NaN en posición + velocidad = "joystick" (`velocity_control`);
- yaw NaN + `yawspeed` = girar a cierta velocidad.

NaN: “no lo uses”

```cpp
#include <limits>
constexpr float kNaN = std::numeric_limits<float>::quiet_NaN();

msg.position = {kNaN, kNaN, kNaN};   // no controles posición
msg.velocity = {2.0f, 0.0f, 0.0f};   // 2 m/s al norte
msg.yaw = kNaN;                      // no fijes el yaw
```

Código 8: Como en goto_mission y velocity_control

- NaN (*Not a Number*) es un valor especial de `float`: el resultado de 0/0.
- Cualquier comparación con NaN da `false`, **incluso** `x == x`.
- Para preguntar: `std::isnan(x)` o `std::isfinite(x)`.

std_msgs: mensajes que no son de PX4

Para pasarse un valor entre nodos nuestros no hace falta `px4_msgs`:

```
float32 data
```

Código 9: ros2 interface show std_msgs/msg/Float32

- `battery_publisher` publica el porcentaje de batería en `/vespo/battery_percent` como `std_msgs/Float32`.
- Cualquier nodo lo lee sin saber nada de PX4.
- Hay `Bool`, `Int32`, `String`, `Float64`... y en `geometry_msgs`, `sensor_msgs` mensajes más grandes (poses, imágenes, scans).

Contenido

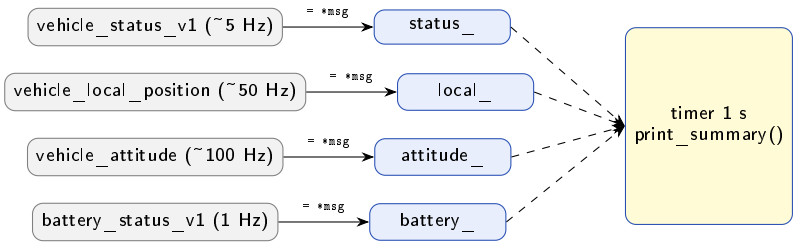
1 PX4 y el x500
2 Cómo habla ROS 2 con PX4
3 Los mensajes de PX4
4 Ejemplos de solo lectura
5 Un topic propio
6 Offboard: nosotros calculamos
7 Comandos: PX4 vuela

Cinco nodos que solo escuchan

Nodo	Lee	Para qué
`telemetry_listener`	estado, posición, GPS, actitud, batería	un resumen cada segundo
`imu_listener`	`sensor_combined`	giroscopio y acelerómetro
`gps_status`	`vehicle_gps_position`	¿ya se puede volar con GPS?
`flight_mode_monitor`	`vehicle_status_v1`, `failsafe_flags`	cada cambio de modo y failsafe
`battery_monitor`	`battery_status_v1`	avisar bajo un % (y opcional RTL/LAND)

No mandan nada a PX4 (salvo `battery_monitor` si se lo piden): se pueden dejar corriendo **mientras** otro nodo vuela, para ver qué está pasando.

El patrón: recibir en callbacks, imprimir en el timer



Cada topic llega a su ritmo; los callbacks solo guardan **la última copia**. El timer lee esas copias y escribe una sola línea por segundo, sin llenar la terminal.

telemetry_listener

Se suscribe a 5 topics y cada segundo imprime:

- **Estado**: armado o no, y el modo (nombre y número).
- **Local**: x, y, altura ($-z$) y velocidad NED.
- **GPS**: lat, lon, alt (AMSL).
- **Actitud**: roll, pitch, yaw en grados (del cuaternión).
- **Batería**: voltaje y %.

ros2 run px4_examples telemetry_listener

```
Estado: desarmado | modo: HOLD (4)
Local: x=0.01 y=-0.02 alt=0.03 m
| vel=(0.00, 0.00, 0.00) m/s
GPS: lat=47.3979711 lon=8.5461637
alt=0.1 m (AMSL)
Actitud: roll=0.1° pitch=-0.2° yaw=90.0°
Batería: 16.20 V | 100 %
```

telemetry_listener: ¿llegaron datos?

```cpp
void print_summary() {
  // Si timestamp sigue en 0 es que todavía no ha llegado ningún mensaje
  // (los mensajes se crean con todos sus campos en 0). Suele indicar que
  // PI4 o el agente no están corriendo, o que el nombre del topic está mal.
  if (status_.timestamp == 0) {
    RCLCPP_WARN(get_logger(), "Sin datos de vehicle_status todavía (¿corre el agente?)");
    return;
  }
  ...
```

Código 10: telemetry_listener.cpp

Truco útil en cualquier nodo: un miembro con `timestamp == 0` significa "nunca llegó un mensaje". Antes de usar la posición, revisen que ya haya llegado (y que `xy_valid` sea `true`).

telemetry_listener: de nav_state a texto

```
const char * nav_state_name(uint8_t s) {
  switch (s) {
    case VehicleStatus::NAVIGATION_STATE_AUTO_LOITER: return "HOLD";
    case VehicleStatus::NAVIGATION_STATE_OFFBOARD:    return "OFFBOARD";
    case VehicleStatus::NAVIGATION_STATE_AUTO_LAND:   return "LAND";
    ...
    default: return "OTRO";   // cualquier otro valor
  }
}
```

Código 11: Una función que traduce el número a un nombre

`const char*` es texto fijo de C: lo que espera `%s` en `RCLCPP_INFO`. Cada `case` termina con `return`, así que no hace falta `break`.

imu_listener

Lee `/fmu/out/sensor_combined` (cientos de Hz) y muestra:

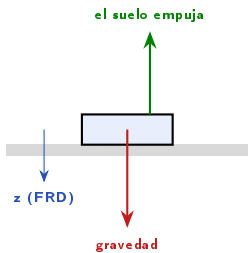
- **Gyro**: velocidad de giro en x, y, z (rad/s y °/s).
- **Accel**: aceleración en x, y, z (m/s^2).
- La frecuencia a la que llegan los mensajes.

Prueben girar el dron en Gazebo (o despegarlo con otro nodo) y vean cómo cambian.

ros2 run px4_examples imu_listener

```
Gyro [rad/s]: x=+0.001 y=-0.002 z=+0.000
(deg/s: +0.1 -0.1 +0.0)
Accel [m/s²]: x=+0.012 y=-0.008
z=-9.806 | 250 Hz
```

¿Por qué el acelerómetro marca -9.81 quieto?



- Un acelerómetro mide la **fuerza específica**: todo menos la gravedad.
- Quieto en el suelo, lo único que siente es al suelo empujando **hacia arriba**.
- En FRD z apunta hacia abajo, así que arriba es z **negativa**: $(0, 0, -9.81)$.
- En caída libre marcaría $(0, 0, 0)$.

gps_status

Lee el GPS **crudo** y dice si ya se puede volar con él.
Revisa:

- `fix_type` ≥ 3 (3D),
- `satellites_used` $\geq$ `min_satellites` (6),
- `eph` $\leq$ `max_eph` (3.0 m).

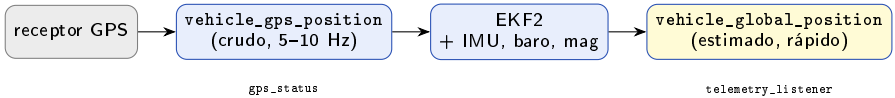
```
ros2 run px4_examples gps_status --ros-args \
  -p min_satellites:=8 -p max_eph:=2.0
```

ros2 run px4_examples gps_status

```
3D | 10 sat | eph 0.9 m epv 1.4 m
| hdop 0.8 | 47.3979711, 8.5461637
GPS listo para volar
```

En el dron real, adentro de un edificio verán “Todavía no:” seguido de lo que falta.

GPS crudo contra posición estimada



Para **volar** se usa la estimada; para **diagnosticar** el GPS (satélites, interferencia), la cruda.

flight_mode_monitor

Imprime **solo cuando algo cambia**:

- armado / desarmado,
- el modo (y si PX4 está en otro distinto del que pidió el usuario),
- failsafe activo / terminado,
- chequeos de pre-vuelo OK o fallando,
- las banderas de `failsafe_flags` activas.

ros2 run px4_examples flight_mode_monitor

```
DESARMADO
Modo: Hold
Chequeos de pre-vuelo: OK, se puede armar
ARMADO
Modo: Hold -> Offboard
Modo: Offboard -> Hold
FAILSAFE ACTIVO (modo Hold)
Banderas de failsafe:
señal offboard perdida
```

flight_mode_monitor: comparar con lo anterior

```
void on_status(const VehicleStatus & s) {
  if (first_ || s.nav_state != last_.nav_state) {
    RCLCPP_INFO(get_logger(), "Modo: %s -> %s", mode_name(last_.nav_state),
                mode_name(s.nav_state));
    // Si el modo activo no es el que pidió el usuario, PX4 lo cambió solo.
    if (s.nav_state != s.nav_state_user_intention) { ... }
  }
  last_ = s;          // guardar para la próxima comparación
  first_ = false;
}
```

Código 12: flight_mode_monitor.cpp (recortado)

Guardar el mensaje anterior y comparar es la forma de detectar **eventos** (cambios) en lugar de imprimir el estado una y otra vez.

Úsenlo para entender qué hace PX4



Dejen `flight_mode_monitor` corriendo en una terminal y en otra:

- Corran `takeoff_mission` y luego Ctrl+C: verán **Offboard** → **failsafe**.
- Corran `orbit_command`: **Takeoff** → **Hold** → **Orbit** → **Return** → **Land**.
- Cambien de modo desde QGroundControl.

Cuando un nodo "no hace nada", casi siempre es porque PX4 está en un modo distinto del que creían. Este monitor lo dice en una línea.

battery_monitor

Lee la batería, avisa bajo `low_percent` y, si se le pide, actúa:

action	Qué hace
none	solo avisa
rtl	NAV_RETURN_TO_LAUNCH
land	NAV_LAND ahí mismo

Solo actúa si el dron **está armado**.

```
battery_monitor -p low_percent:=40.0 -p action:=rtl
```

```
85.0% | 15.80 V (3.95 V/celda, 4S) | -1.0 A | OK
BATERÍA BAJA: 39.8% (umbral 40%)
Regresando a casa (RTL)
```

El failsafe de PX4 contra el nuestro

PX4 ya tiene failsafe de batería (`BAT_LOW_THR`, `BAT_CRIT_THR`, `COM_LOW_BAT_ACT`). ¿Para qué otro?

- Avisar **antes**, con un umbral propio de la misión.
- Que la **misión decida**: cortar, guardar dónde iba, ir a cargar (como la estación de carga del Nivel 3 de los retos).

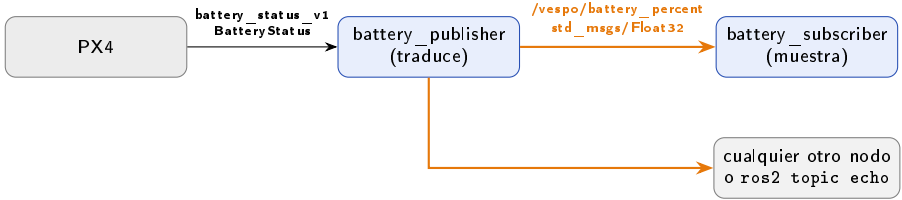
Ojo

En SITL `current_a` es -1 (no hay sensor de corriente simulado) y la batería baja del 100 al 50 % en 30 s armado. No usen esos números para estimar el vuelo real.

Contenido

1. PX4 y el x500
2. Cómo habla ROS 2 con PX4
3. Los mensajes de PX4
4. Ejemplos de solo lectura
5. Un topic propio
6. Offboard: nosotros calculamos
7. Comandos: PX4 vuela

battery_publisher y battery_subscriber



- Un nodo **traduce** lo que dice PX4 a algo simple: solo el porcentaje.
- Cualquier otro nodo lo usa **sin saber nada** de `px4_msgs`: una interfaz, la misión, un registro...
- Es el patrón para pasarse datos entre nodos nuestros.

El publisher

```
// Publisher con QoS por defecto: rclcpp::QoS(10) = reliable, cola de 10.
percent_pub_ = create_publisher<Float32>("/vespo/battery_percent", 10);

auto px4_qos = rclcpp::QoS(1).best_effort().transient_local();
battery_sub_ = create_subscription<BatteryStatus>(
  "/fmu/out/battery_status_v1", px4_qos,
  [this](const BatteryStatus::SharedPtr msg) {
    // remaining = -1 cuando PX4 aún no sabe: no publicamos basura.
    if (!msg->connected || msg->remaining < 0.0f) {
      return;
    }
    Float32 out;
    out.data = msg->remaining * 100.0f;   // 0..1 -> 0..100 %
    percent_pub_->publish(out);
  });
```

Código 13: battery_publisher.cpp

Publica **dentro** del callback: cada vez que llega una batería de PX4 (1 Hz), sale un porcentaje. No hace falta timer.

Las decisiones de diseño

Decisión	Por qué
Nombre `/vespo/...`	Convención del equipo: nuestros topics no se confunden con los de PX4 (`/fmu/...`) ni de otros paquetes.
`std_msgs/Float32`	Un mensaje estándar con un solo campo (`data`): no hace falta crear uno.
QoS por defecto (10)	Entre nodos de ROS 2 no hace falta el best_effort de PX4: reliable, cola de 10.
Filtrar `remaining < 0`	-1 = "PX4 no sabe": mejor no publicar que publicar basura.

El subscriber

```
percent_sub_ = create_subscription<Float32>(
  "/vespo/battery_percent", 10,
  [this](const Float32::SharedPtr msg) {on_percent(msg->data);});

void on_percent(float percent) {
  // Barra de 20 caracteres: cada # vale 5 %.
  const int filled = static_cast<int>(percent / 5.0f + 0.5f);
  std::string bar;
  for (int i = 0; i < 20; i++) {
    bar += i < filled ? "#" : ".";
  }
  if (percent < low_percent_) {
    RCLCPP_WARN(get_logger(), "[%s] %5.1f %% BAJA", bar.c_str(), percent);
  } else {
    RCLCPP_INFO(get_logger(), "[%s] %5.1f %%", bar.c_str(), percent);
  }
}
```

Código 14: battery_subscriber.cpp

Detalles del subscriber

- El callback llama a un método (`on_percent`) con **solo el número**: el resto del código no sabe de mensajes.
- `+ 0.5f` antes de convertir a `int` **redondea** (sin él, `static_cast` trunca: 9.9 → 9).
- `bar.c_str()`:`%s` pide texto de C (`const char*`); `c_str()` lo saca de un `std::string`.

ros2 run px4_examples battery_subscriber

```
[####################] 100.0%
################...
84.5%
###.............
25.0%  BAJA
```

Probarlo sin PX4

```
ros2 run px4_examples battery_subscriber            # terminal 1
ros2 topic pub /vespo/battery_percent std_msgs/msg/Float32 "{data: 15.0}"   # terminal 2
ros2 topic echo /vespo/battery_percent              # ver lo que pasa por el topic
ros2 topic info /vespo/battery_percent              # quién publica y quién escucha
```

Como el subscriber solo depende del topic, se puede probar **sin simulación**: publiquen valores a mano para ver el aviso de batería baja. Así se prueban las partes de una misión por separado.

Ejercicios

1. Agreguen al publisher un segundo topic `/vespo/battery_voltage` con el voltaje.
2. Hagan que el subscriber avise **una sola vez** al cruzar el umbral (no en cada mensaje).
3. Publiquen la altura actual (`-z` de `vehicle_local_position_v1`) en `/vespo/altitude` y grafíquenla con `rqt_plot`.
4. Cambien el QoS del subscriber a `best_effort`: ¿sigue funcionando? ¿Y si el que cambia es el publisher?

Contenido

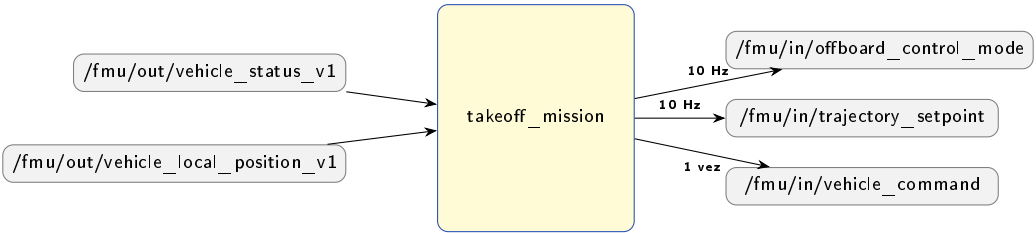
1. PX4 y el x500
2. Cómo habla ROS 2 con PX4
3. Los mensajes de PX4
4. Ejemplos de solo lectura
5. Un topic propio
6. Offboard: nosotros calculamos
7. Comandos: PX4 vuela

Los cinco ejemplos de offboard

Nodo	Qué vuela	Controla con	Lo nuevo
`takeoff_mission`	sube a 5 m y se queda	posición	el heartbeat
`square_mission`	cuadrado de 5 m y aterriza	posición	máquina de estados, waypoints
`orbit_mission`	círculo mirando al centro	posición + velocidad	parámetros, feedforward
`figure_eight`	un 8 en función del tiempo	+ aceleración	derivadas, yaw de avance
`velocity_control`	tramos de velocidad	velocidad	NaN, marco del cuerpo

Todos publican tres cosas: `OffboardControlMode` y `TrajectorySetpoint` **cada ciclo**, y `VehicleCommand` **una vez** para entrar a offboard y armar.

El grafo de un nodo offboard



takeoff_mission: el ciclo completo

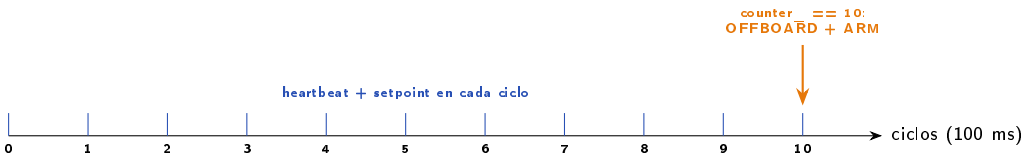
```cpp
// --- 1) Heartbeat: "sigo vivo y voy a controlar por POSICIÓN"
OffboardControlMode offboard_msg{};
offboard_msg.position = true;
offboard_msg.timestamp = now_us();
offboard_pub_->publish(offboard_msg);

// --- 2) Setpoint: z = -5 significa 5 m ARRIBA (NED)
TrajectorySetpoint setpoint_msg{};
setpoint_msg.position = {0.0f, 0.0f, -5.0f};
setpoint_msg.yaw = 0.0f;                  // rad; 0 = mirando al norte
setpoint_msg.timestamp = now_us();
setpoint_pub_->publish(setpoint_msg);

// --- 3) Tras ~1 s de setpoints: pedir OFFBOARD y ARMAR (una sola vez)
if (counter_ == 10) {
  send_command(VehicleCommand::VEHICLE_CMD_DO_SET_MODE, 1.0f, 6.0f);     // 6 = OFFBOARD
  send_command(VehicleCommand::VEHICLE_CMD_COMPONENT_ARM_DISARM, 1.0f);  // 1 = armar
}
```

Código 15: takeoff_mission.cpp, loop() a 10 Hz

¿Por qué esperar 1 s antes de pedir offboard?



- PX4 **rechaza** el cambio a offboard si no le están llegando setpoints **ya**: no sabría qué hacer en el primer instante.
- Por eso: primero ~10 ciclos de setpoints, después los comandos.
- Y nunca se deja de publicar: si el nodo se cae (Ctrl+C), PX4 entra en **failsafe** (`COM_OBL_RC_ACT`).

send_command(): el ayudante de todos

```cpp
void send_command(uint32_t command, float p1 = 0.0f, float p2 = 0.0f) {
  VehicleCommand msg{};
  msg.command = command;
  msg.param1 = p1;
  msg.param2 = p2;
  msg.target_system = 1;       // el dron (MAVLink system id)
  msg.target_component = 1;   // el autopiloto
  msg.source_system = 1;
  msg.source_component = 1;
  msg.from_external = true;    // viene de fuera de PX4 (de ROS 2)
  msg.timestamp = now_us();
  command_pub_->publish(msg);
}
```

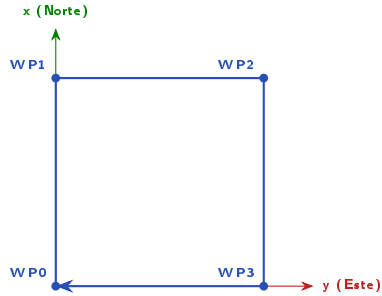
Código 16: Igual en todos los ejemplos

`float p1 = 0.0f`: **valor por defecto**. `send_command(VEHICLE_CMD_NAV_LAND)` lo llama con `p1 = p2 = 0`.

square_mission: waypoints y estados

```
enum class State { WARMUP, MISSION,
                   LANDING, DONE };

std::vector<std::array<float, 3>>
  waypoints_ = {
    {0.0f, 0.0f, -5.0f},   // WP0
    {5.0f, 0.0f, -5.0f},   // WP1
    {5.0f, 5.0f, -5.0f},   // WP2
    {0.0f, 5.0f, -5.0f},   // WP3
    {0.0f, 0.0f, -5.0f}};  // WP4
```



En MISSION manda el waypoint actual; cuando `reached()` da `true`, pasa al siguiente. Al terminar la lista: `NAV_LAND`.

square_mission: ¿ya llegué?

```
bool reached(const std::array<float, 3> & target) {
  float dx = position_.x - target[0];
  float dy = position_.y - target[1];
  float dz = position_.z - target[2];
  float distance = std::sqrt(dx*dx + dy*dy + dz*dz);
  return distance < 0.3f;
}
```

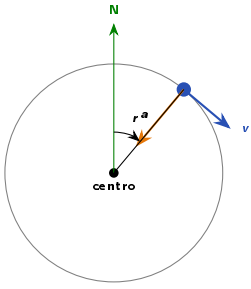
Código 17: square_mission.cpp

- Distancia en 3D entre el dron y el waypoint.
- **0.3 m** de tolerancia: si es muy chica, el dron puede no "llegar" nunca por el ruido.
- En `LANDING` ya no publica setpoints: PX4 está en `AUTO_LAND`.

square_mission

```
Offboard activo, iniciando mision
Llegue al WP 0
Llegue al WP 1
...
Mision completa, aterrizando
```

orbit_mission: un círculo calculado por nosotros



- Ángulo a desde el norte, creciendo hacia el este (horario visto desde arriba).
- Punto: $(c_x + r\cos a,\ c_y + r\sin a)$.
- Cada ciclo: $a \mathrel{+}= \omega\,\Delta t$, con $\omega = v/r$ y $\Delta t = 0.1$ s.
- Velocidad (feedforward): $(-v\sin a,\ v\cos a)$, tangente al círculo.
- Yaw: mirando al centro.

orbit_mission: el estado ORBIT

```
case State::ORBIT: {
  // Las llaves {} crean un bloque: hacen falta para declarar variables
  // (omega) dentro de un case.
  const double omega = speed_ / radius_;
  // Derivada de (r cos a, r sin a): la velocidad es tangente al círculo.
  vx = -speed_ * std::sin(angle_);
  vy = speed_ * std::cos(angle_);
  angle_ += omega * kDt;

  if (angle_ >= 2.0 * M_PI * laps_) {
    send_command(VehicleCommand::VEHICLE_CMD_NAV_LAND);
    state_ = State::LANDING;
  }
  break;
}
```

Código 18: orbit_mission.cpp

`static constexpr double kDt = 0.1;`: el periodo del timer, como constante con nombre.

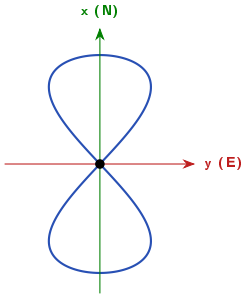
Parámetros de orbit_mission

Parámetro	Qué	Defecto
`center_x`	m al norte	0.0
`center_y`	m al este	0.0
`altitude`	m	5.0
`radius`	m	5.0
`speed`	m/s	2.0
`laps`	vueltas (entero)	2

Pruébenlo con distintos valores **sin recompilar**:

- `-p radius:=8.0 -p speed:=3.0`
- `-p speed:=8.0`: ¿qué pasa con el error de seguimiento?
- `-p laps:=2.5`: **error**, `laps` es entero.

figure_eight: una curva en función del tiempo



La lemniscata de Gerono, con $w = 2\pi/T$:

$$x(t) = A\sin(wt) \qquad y(t) = \tfrac{A}{2}\sin(2wt)$$

- En $t = 0$ pasa por el origen: empieza donde despegó.
- T (`period`) = segundos por cada 8; A (`size`).
- Velocidad máxima $\approx A \cdot 2\pi/T$: con 5 m y 20 s, ~1.6 m/s.

Derivar a mano: velocidad y aceleración

Con $\frac{d}{dt}\sin(wt) = w\cos(wt)$ y $\frac{d}{dt}\cos(wt) = -w\sin(wt)$:

$$v_x = A\,w\cos(wt) \qquad v_y = A\,w\cos(2\,wt)$$
$$a_x = -A\,w^2\sin(wt) \qquad a_y = -2A\,w^2\sin(2wt)$$

- Se mandan **las tres**: posición, velocidad y aceleración.
- Sin feedforward, PX4 corrige *después* de ver el error: se queda atrás de la curva.
- El nodo imprime el error de seguimiento: compárenlo quitando la velocidad.

Yaw de avance: $\psi = \text{atan2}(v_y, v_x)$. La nariz apunta hacia donde se mueve, como un coche.

figure_eight: struct y método const

```cpp
// Posición, velocidad y aceleración de la curva en el instante t.
struct Sample { float x, y, vx, vy, ax, ay; };

Sample curve(double t) const {          // const: no cambia el objeto
  const double w = 2.0 * M_PI / period_;
  const double a = size_;
  return Sample{
    static_cast<float>(a * std::sin(w * t)),
    static_cast<float>(a / 2.0 * std::sin(2.0 * w * t)),
    ...
  };
}
```

Código 19: figure_eight.cpp

- `struct`: agrupar varios valores con nombre (como una clase con todo público).
- `const` después de los paréntesis: el método **promete** no modificar miembros.
- 20 Hz (timer de 50 ms): una trayectoria que cambia rápido se sigue mejor con más setpoints.

velocity_control: como un joystick

Un plan de **tramos**: “durante 4 s, 2 m/s al norte”, “6 s girando a 0.5 rad/s”...

- `OffboardControlMode`: `velocity = true`, `position = false`.
- Posición en **NaN**: “no controles posición”.
- Yaw en NaN y `yawspeed` con valor: girar a cierta velocidad.

Ojo

En velocidad el dron **no se queda en un punto**. Si mandas 0 se frena, pero con viento se va moviendo poco a poco: nadie corrige la posición.

velocity_control: el plan

```
//   nombre                        fwd   right down  yawspeed  s     cuerpo
std::vector<Leg> legs_ = {
  {"norte a 2 m/s",                2.0f, 0.0f, 0.0f, 0.0f,    4.0, false},
  {"este a 2 m/s",                 0.0f, 2.0f, 0.0f, 0.0f,    4.0, false},
  {"girar en el lugar",            0.0f, 0.0f, 0.0f, 0.5f,    6.0, false},
  {"adelante (hacia donde mira)",  2.0f, 0.0f, 0.0f, 0.0f,    4.0, true},
  {"adelante girando (curva)",     2.0f, 0.0f, 0.0f, 0.4f,    6.0, true},
  {"bajar a 0.5 m/s",              0.0f, 0.0f, 0.5f, 0.0f,    3.0, false},
  {"frenar",                       0.0f, 0.0f, 0.0f, 0.0f,    2.0, false},
};
```

Código 20: velocity_control.cpp: cambia, quita o agrega tramos

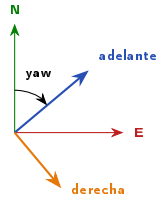
Un `std::vector` de `struct Leg`, inicializado con llaves. `cuerpo = true`: las velocidades son “adelante / derecha” del dron, no norte / este.

Del cuerpo a NED

```
// Rotar (adelante, derecha) por el heading (yaw) del dron:
//   norte = adelante*cos(yaw) - derecha*sin(yaw)
//   este  = adelante*sin(yaw) + derecha*cos(yaw)
const float yaw = position_.heading;
vx = leg.forward * std::cos(yaw) - leg.right * std::sin(yaw);
vy = leg.forward * std::sin(yaw) + leg.right * std::cos(yaw);

setpoint_msg.position = {kNaN, kNaN, kNaN};   // si fuera {0,0,0}, ¡volvería al origen!
setpoint_msg.velocity = {vx, vy, leg.down};
setpoint_msg.yaw = kNaN;
setpoint_msg.yawspeed = leg.yawspeed;
```

Código 21: velocity_control.cpp



Es lo que se usa para **seguir algo con la cámara**: "avanza hacia donde miras".

El despegue se cierra con la altura, no con el tiempo

Tentación: "sube 5 s a 1 m/s" = 5 m. En la práctica **no**:

- PX4 tarda en arrancar motores y hace una rampa de despegue.
- Con el tiempo como criterio, el dron se queda casi en el suelo cuando empieza el siguiente tramo.

`velocity_control` sube a 1 m/s **hasta medir** la altura objetivo; luego empiezan los tramos.

Regla general: las transiciones de una misión se disparan con **lo que se mide** (posición, altura, modo), no con lo que *debería* haber pasado.

Seguridad en offboard: soltar el control

Todos los ejemplos de offboard revisan en cada ciclo:

`state_ != WARMUP && !in_offboard()` $\Rightarrow$ abortar

- Si el piloto cambia de modo desde QGC o el control remoto, o hay un failsafe, el nodo **deja de mandar** y termina.
- Así el nodo nunca "pelea" contra el piloto.

orbit_mission

```
Fuera de OFFBOARD (nav=4):
misión abortada
```

Contenido

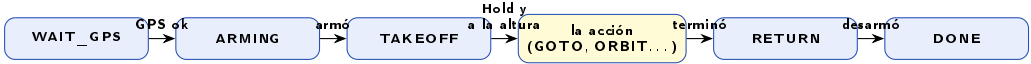
1. PX4 y el x500
2. Cómo habla ROS 2 con PX4
3. Los mensajes de PX4
4. Ejemplos de solo lectura
5. Un topic propio
6. Offboard: nosotros calculamos
7. Comandos: PX4 vuela

Los seis ejemplos con comandos

Nodo	Qué hace	Comandos
`orbit_command`	despega, orbita, regresa	TAKEOFF, DO_ORBIT, RETURN
`goto_mission`	el cuadrado por coordenadas GPS	TAKEOFF, DO_REPOSITION, RETURN
`return_home`	se aleja y muestra las fases de Return	TAKEOFF, DO_REPOSITION, RETURN
`change_speed`	cambia la velocidad en pleno vuelo	DO_REPOSITION, DO_CHANGE_SPEED
`set_home`	mueve "casa" y aterriza ahí	DO_SET_HOME, RETURN
`motor_test`	gira un motor, desarmado	ACTUATOR_TEST

Sin heartbeat ni setpoints: el nodo manda **una orden** y espera a que PX4 la cumpla (mirando el modo, la posición o el ack). Si el nodo se cae, PX4 sigue en su modo.

El patrón: mandar, esperar, mandar



- Cada estado manda su comando **una vez** al entrar y luego **solo mira**.
- "Terminó" se mide: el modo cambió a Hold, la distancia al punto es pequeña, desarmó...

send_command con los 7 parámetros

```
void send_command(uint32_t command, float p1 = 0.0f, float p2 = 0.0f, float p3 = 0.0f,
  float p4 = 0.0f, double p5 = 0.0, double p6 = 0.0, float p7 = 0.0f)
{
  VehicleCommand msg{};
  msg.command = command;
  msg.param1 = p1;  msg.param2 = p2;  msg.param3 = p3;  msg.param4 = p4;
  msg.param5 = p5;  msg.param6 = p6;  msg.param7 = p7;
  ...
  command_pub_->publish(msg);
}
```

Código 22: goto_mission.cpp: param5 y param6 son double

En MAVLink, param5 y param6 suelen ser latitud y longitud: en `float` se perdería ~1 m. Por eso en `VehicleCommand` son `float64`.

Leer el ack: ¿por qué no me hizo caso?

```
ack_sub_ = create_subscription<VehicleCommandAck>(
  "/fmu/out/vehicle_command_ack", qos,
  [this](const VehicleCommandAck::SharedPtr msg) {
    // Solo avisa de rechazos, y una vez por cada respuesta distinta.
    const bool repeated = msg->command == last_ack_.command && msg->result == last_ack_.result;
    last_ack_ = *msg;
    if (!repeated && msg->result != VehicleCommandAck::VEHICLE_CMD_RESULT_ACCEPTED) {
      RCLCPP_WARN(get_logger(), "PX4 rechazó el comando %u (result=%u)",
                  msg->command, msg->result);
    }
  });
```

Código 23: goto_mission.cpp

Sin esto, un comando rechazado se ve igual que uno que "no hizo nada". Con esto, la terminal dice cuál y por qué (con la tabla de `result`).

Armar en modo Hold, y reintentar

```
if (!armed) {
  if (every_second()) {
    // Primero a Hold (AUTO_LOITER): en algunos modos, como Land (el modo
    // en que queda tras aterrizar), PX4 no deja armar.
    // DO_SET_MODE: param1 = 1 (modo custom), param2 = 4 (AUTO), param3 = 3 (LOITER).
    if (status_.nav_state != VehicleStatus::NAVIGATION_STATE_AUTO_LOITER) {
      send_command(VehicleCommand::VEHICLE_CMD_DO_SET_MODE, 1.0f, 4.0f, 3.0f);
    } else {
      send_command(VehicleCommand::VEHICLE_CMD_COMPONENT_ARM_DISARM, 1.0f);
    }
  }
  break;
}
```

Código 24: goto_mission.cpp, estado ARMING

`every_second()` manda como mucho un comando por segundo: si PX4 todavía no está listo, se reintenta sin inundarlo.

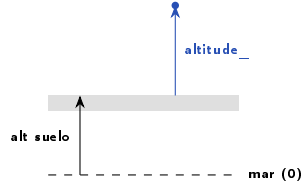
NAV_TAKEOFF: la altitud es AMSL

```
// Al tener GPS, guardar el punto de despegue en las dos coordenadas:
home_lat_ = global_.lat;  home_lon_ = global_.lon;  home_alt_ = global_.alt;

send_command(VehicleCommand::VEHICLE_CMD_NAV_TAKEOFF, kNaN, kNaN, kNaN, kNaN,
             home_lat_, home_lon_, home_alt_ + altitude_);   // AMSL = suelo + altura
```

Código 25: goto_mission.cpp

- Los comandos usan altitud sobre el **nivel del mar** (AMSL), no sobre el suelo.
- `global_.alt` con el dron en el suelo = la altitud del suelo.
- Los params que no aplican van en NaN.



goto_mission: el cuadrado por GPS

Los mismos waypoints que `square_mission`, en metros {norte, este}, convertidos a lat/lon:

$$\text{lat} = \text{lat}_0 + \frac{N}{R} \cdot \frac{180}{\pi} \qquad \text{lon} = \text{lon}_0 + \frac{E}{R\cos(\text{lat}_0)} \cdot \frac{180}{\pi}$$

$R = 6\,371\,000$ m. Los meridianos se juntan hacia los polos: de ahí el cos.

Y para ir: `DO_REPOSITION`

- p1 = velocidad (m/s; -1 = la de PX4)
- p2 = 1 (obligatorio: "cambia a Hold")
- p4 = yaw (NaN = no girar)
- p5, p6, p7 = lat, lon, alt AMSL

PX4 vuela con su propia aceleración y frenado, y **se queda en Hold** al llegar.

Offboard contra comandos: el mismo cuadrado

	`square_mission` (offboard)	`goto_mission` (comandos)
Coordenadas	NED local (m)	GPS (lat, lon, alt AMSL)
Qué se publica	setpoint a 10 Hz	un comando por waypoint
Velocidad	la decide PX4 hacia el setpoint	parámetro `speed` del comando
Si el nodo se cae	failsafe	se queda en Hold
Cómo termina	`NAV_LAND`	`NAV_RETURN_TO_LAUNCH`

orbit_command: el modo Orbit de PX4

```
// DO_ORBIT: param1 = radio, param2 = velocidad, param3 = hacia dónde mira
// (0 = al centro), param5/6 = centro (lat/lon), param7 = altitud (NaN = la actual).
send_command(VehicleCommand::VEHICLE_CMD_DO_ORBIT,
             static_cast<float>(radius_), static_cast<float>(speed_),
             VehicleCommand::ORBIT_YAW_BEHAVIOUR_HOLD_FRONT_TO_CIRCLE_CENTER, kNaN,
             center_lat, global_.lon, kNaN);
```

Código 26: orbit_command.cpp

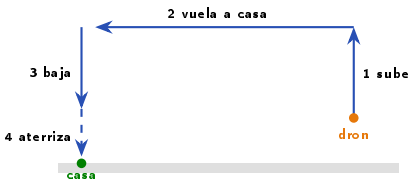
Comparen con `orbit_mission`: allá calculábamos cada punto del círculo; aquí **una línea**.

- Menos control fino (no podemos cambiar la forma).
- El nodo cuenta vueltas con el tiempo ($2\pi r/v$ por vuelta) y luego manda Return.

flight_mode_monitor

```
Modo: Hold -> Takeoff
Modo: Takeoff -> Hold
Modo: Hold -> Orbit
Modo: Orbit -> Return
Modo: Return -> Land
```

return_home: las fases de Return



1. Sube a `RTL_RETURN_ALT` si está más bajo (30 m en el x500 de SITL), recortado por un **cono** sobre casa (`RTL_CONE_ANG`).
2. Vuela en línea recta a casa.
3. Baja a `RTL_DESCEND_ALT` (10 m) y espera `RTL_LAND_DELAY`.
4. Aterriza y desarma solo.

return_home: deducir la fase

```
const char * phase;
if (status_.nav_state != VehicleStatus::NAVIGATION_STATE_AUTO_RTL) {
  phase = "ya no está en Return (¿alguien cambió el modo?)";
} else if (dist > 2.0 && local_.vz < -0.3) {
  phase = "subiendo a la altura de retorno";      // vz < 0: subiendo (NED)
} else if (dist > 2.0) {
  phase = "volando a casa";
} else {
  phase = "sobre casa, bajando";
}
```

Código 27: return_home.cpp

PX4 no publica “en qué fase de Return voy”: se deduce de lo que sí publica (modo, distancia a casa, velocidad vertical).

change_speed: DO_CHANGE_SPEED

1. Despega y va hacia un punto a `distance` m al norte a `slow` m/s.
2. A un tercio del camino manda `DO_CHANGE_SPEED` a `fast` m/s.
3. Al llegar, Return.

p1	1 = velocidad sobre el suelo
p2	m/s (−1 = la de por defecto)
p3	acelerador % (−1 = no cambiar)

Ojo

Afecta a los modos automáticos de PX4 (Hold/Go to, Mission). En **offboard no hace nada**: ahí la velocidad la decide tu setpoint.

set_home: mover “casa”

`DO_SET_HOME` tiene dos formas:

- p1 = 1: casa = donde está el dron **ahora**.
- p1 = 0 y p5..p7: casa = esas coordenadas (lat, lon, alt AMSL).

Flujo: arma (PX4 fija casa ahí) → despega → `DO_SET_HOME` → espera a que `home_position.update_count` cambie → Return → aterriza en la casa nueva.

Ojo

`home_position` se publica **solo cuando cambia**; un nodo que arranca después no recibe el último valor. Por eso `set_home` no espera tener casa antes de armar: al armar llega.

motor_test: probar los motores

Lo mismo que la pestaña “Motores” de QGC, desde ROS 2. Sirve para revisar en el dron real:

- que cada motor gire,
- que el **motor 1** sea el que crees (si están cruzados, el dron se voltea al despegar),
- el sentido de giro de cada uno.

`ACTUATOR_TEST`: p1 = 0..1, p2 = segundos, p5 = motor (1..8).

¡SIN HÉLICES!

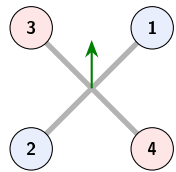
Aunque esté **desarmado**, el motor **sí gira**. PX4 solo lo acepta desarmado y con `COM_MOT_TEST_EN` = 1. El nodo limita el empuje a 30 %.

motor_test: uno o todos

```
ros2 run px4_examples motor_test                                    # motor 1, 15 %, 2 s
ros2 run px4_examples motor_test --ros-args -p motor:=3 -p throttle:=0.2 -p seconds:=3.0
ros2 run px4_examples motor_test --ros-args -p motor:=0             # todos, en orden
```

Con `motor:=0` recorre del 1 al 4 uno por uno.
Comparen con el diagrama:

- 1 adelante derecha, antihorario
- 2 atrás izquierda, antihorario
- 3 adelante izquierda, horario
- 4 atrás derecha, horario



¿Cuándo offboard y cuándo comandos?

Usen **offboard** cuando...	Usen **comandos** cuando...
la trayectoria es propia (estrella, hélice, un 8)	basta "ir a un punto", "orbitar", "regresar"
hay que reaccionar a un sensor 10–20 veces por segundo	el nodo puede caerse sin poner en riesgo el vuelo
se sigue algo que se mueve	se vuela por GPS con los modos probados de PX4

Se pueden mezclar: despegar con Takeoff, hacer la parte fina en offboard y terminar con Return.

Créditos de las imágenes



- **Fotos del X500 y la Pixhawk 6C, capturas de QGroundControl y el diagrama de uXRCE-DDS**: documentación de PX4 (`docs.px4.io`), © PX4 / Dronecode y colaboradores, licencia **CC BY 4.0** (`creativecommons.org/licenses/by/4.0`). Solo se redimensionaron.
- **Renders del x500**: propios, con Gazebo Harmonic sobre el modelo `x500` de PX4.